Qwen

From Raw Data to Automated Execution: An Actionable Blueprint for Integrating Quant Libraries into the TITAN Trading Pipeline

This report outlines a comprehensive architectural design for an integration layer that incorporates a suite of specified open-source quantitative finance libraries into the existing institutional trading pipeline, TITAN . The objective is to create a full research-to-execution system capable of operating on global market data, including stocks, foreign exchange, cryptocurrencies, and commodities, with a strict historical data boundary dating back to the year 2000 . The architecture is designed to be action-oriented, focusing on mapping each library to a specific function within the pipeline, defining clear data flows, and leveraging pre-existing components such as GARCH volatility forecasts and Kalman trend estimates where appropriate . The analysis prioritizes direct implementability over theoretical exploration, providing a pragmatic blueprint for building a robust, end-to-end quantitative trading framework.

Foundational Data Management and Ingestion Strategy

The successful integration of any quantitative library is contingent upon the quality, consistency, and accessibility of its input data. For the TITAN integration layer, establishing a robust data management and ingestion strategy is the foundational bedrock upon which all subsequent analytical and execution stages will be built. The project's scope demands a system that ingests and processes multi-asset, multi-market data, spanning global equities, forex, crypto, and commodities, with a non-negotiable historical depth beginning in the year 2000 . This requirement necessitates a dedicated module responsible for sourcing, cleaning, aligning, and storing data in a standardized format that serves as a single source of truth for the entire system. Without this foundation, the outputs of sophisticated libraries like `Alphalens` or `Riskfolio-Lib` would be unreliable, and backtests conducted with `VectorBT` or `Backtrader` could produce misleading results due to data inconsistencies.

The first critical task for the data management module is sourcing raw data from diverse origins. The provided context highlights a range of potential sources, from specialized commercial providers to open-source community platforms. BMLL, for instance, is identified as a leading specialist in historical data, offering Level 1, 2, and 3 market data for Equities, ETFs, Futures, and Options, collected directly from global exchanges [57] [86]. Similarly, Cboe Market Data Services provides high-quality, real-time data streams essential for live trading applications [53]. While these premium services offer reliability and breadth, the project's reliance on open-source tools suggests a more hybrid approach may be most effective. OpenBB Terminal emerges as a central candidate for managing data access, acting as a unified interface to various sources [48]. Its documentation indicates capabilities for retrieving both historical and real-time data, including exchange rates [15], and it supports integrations with other data providers like Tushare for accessing Chinese markets [14]. Furthermore, the Qlib framework itself supports more than 30 different data sources, covering stocks, cryptocurrencies, and forex, demonstrating the viability of a modular data ingestion strategy [59]. Therefore, the integration layer should architect the data management module to support multiple connectors, allowing for flexibility in sourcing data based on cost, coverage, and quality requirements.

Once sourced, the raw data must undergo a rigorous cleaning and processing phase. Financial time-series data is notoriously messy, often containing irregular sampling intervals, missing values, and structural breaks [88] [89]. The integration layer must include logic to handle these issues systematically. For example, when dealing with cryptocurrency data from APIs like Binance, a websocket connection is the preferred method for receiving tick-by-tick updates, ensuring minimal latency and avoiding gaps in the data stream [49]. For historical data, handling missing values is a key challenge; while some techniques exist for imputation, studies show that naive methods can corrupt underlying data structures, suggesting a cautious approach is needed [88]. Irregularly sampled data sets with up to 15% missing data can potentially be repaired for analysis, but this requires careful consideration of the methodology [89]. The output of this cleaning process must be a consistent, aligned dataset. All assets across the four classes (stocks, forex, crypto, commodities) must have their price and volume data synchronized to a common timestamp basis (e.g., minute-level or daily). This alignment is a prerequisite for feeding the data into any backtesting engine or portfolio optimizer. The 'from 2000' constraint is a critical filter applied during this stage; datasets must be verified to meet this temporal boundary before being passed to any analytical tool .

A significant portion of the project's requirements involves incorporating real-time data alongside the extensive historical record . This dual requirement points towards a hybrid data architecture. For historical analysis, batch processing workflows are sufficient.

However, for live trading and execution learning, a streaming architecture is necessary. Technologies like Apache Kafka are commonly used to build robust, high-throughput data pipelines that can handle real-time financial data streams [37]. Such a system would involve data producers (APIs from brokers or exchanges) pushing ticks and bars to a Kafka topic, which is then consumed by data processing consumers within the TITAN pipeline. These consumers would perform real-time cleaning, feature extraction, and signal generation. The output of this stream could feed directly into the execution protocol or be stored in a low-latency database like Redis or DynamoDB, which can serve as a feature store for immediate inference requests [37]. This warm path architecture separates the real-time data flow from the main processing logic, ensuring production stability [37]. For less demanding tasks, simpler solutions using native websockets or API polling might suffice initially, but the long-term architecture should anticipate the need for a scalable streaming platform.

Finally, the data management module must provide a standardized interface for all downstream components. Instead of requiring each library (e.g., `tsfresh`, `Alphalens`, `VectorBT`) to have its own data loading logic, the integration layer should expose a unified data access API. This API would allow components to request data for a specific set of assets and date range in a consistent format, typically a pandas DataFrame. This abstraction decouples the data source from the analytics, making the system more maintainable and adaptable. For instance, if a user wants to run a factor analysis on global currencies since 2000, the Factor Evaluation stage would call the data manager's `get_historical_prices(['EUR/USD', 'GBP/USD'], start='2000-01-01')` function. The data manager would then orchestrate the retrieval from the appropriate sources (e.g., BMLL for historical FX, a real-time provider for current quotes) and return a clean, ready-to-use DataFrame. This approach ensures that all stages of the pipeline operate on a consistent and reliable data foundation, which is the ultimate prerequisite for building a trustworthy research-to-execution system. The table below summarizes the key considerations for the data management module.

Component	Key Considerations & Tools	Rationale
Data Sourcing	Hybrid approach using OpenBB Terminal, BMLL, Cboe, Qlib, and custom APIs (e.g., ccxt for crypto) [14] [53] [57] [59] [61].	Balances cost, coverage, and the project's focus on open-source tools while ensuring access to high-quality data.
Historical Data Handling	Adherence to the 'from 2000' boundary [27]. Alignment of multi-asset OHLCV data. Use of maintained forks like Zipline-Reloaded for academic compatibility [24].	Ensures compliance with project constraints and provides consistent input formats for backtesting and analysis libraries.
Real-Time Data Streaming	Utilize websockets for crypto/tick data [49] and consider Kafka-based architectures for a scalable, production-grade solution [37].	Provides low-latency data feeds essential for live execution and avoids data gaps that could invalidate strategies.
Data Cleaning & Quality	Address missing values and irregular sampling [88] [89]. Handle structural breaks identified by tests like GSADF [91].	Prevents garbage-in, garbage-out scenarios. Ensures the integrity of statistical analyses performed by libraries like `Alphalens`.
Standardized Interface	Develop a unified data access API to abstract away data sources and formats.	Decouples data ingestion from analytics, improving modularity and maintainability of the overall system.

By meticulously designing this foundational layer, the integration of the specified open-source libraries becomes a matter of connecting analytical modules to a reliable data source, rather than struggling with data acquisition and preparation at every step. This strategic focus on the data bedrock is the first and most critical step toward achieving the project's ambitious goals.

Stage 1: Feature Engineering and Alpha Discovery

With a robust data foundation in place, the next logical stage in the research-to-execution pipeline is to transform raw market data into actionable insights. This two-part stage encompasses systematic feature engineering and rigorous alpha discovery through factor evaluation. The primary objective is to identify statistically significant, predictive signals from the vast information contained in historical and real-time market data. The integration layer maps the `tsfresh` library to the feature engineering role and pairs `Alphalens-Reloaded` and `Pyfolio-Reloaded` with the alpha discovery and performance analysis roles. This combination provides a powerful, modern toolkit for moving from raw data to validated factors, forming the core of the TITAN pipeline's research capability.

The feature engineering stage is powered by `tsfresh`, a Python package designed for the automated extraction of features from time-series data [38] [90]. Its purpose is to systematically calculate a large number of characteristics from sequential data, which can

then be used as inputs for machine learning models or as standalone factors [78]. The integration layer would feed clean, aligned historical data (OHLCV, tick data, etc.) from the foundational data management module into `tsfresh`. The library's strength lies in its ability to generate a comprehensive feature matrix where each row corresponds to an asset at a specific point in time, and each column represents a calculated feature, such as rolling standard deviation, autocorrelation at different lags, or spectral entropy [31]. This process is highly configurable and can handle time series of varying lengths, which is crucial given the different listing dates and data availability across global stocks, forex pairs, and cryptocurrencies [62]. Furthermore, `tsfresh` supports operations like rolling windows, which can be used to construct hierarchical, multi-timeframe feature pipelines—a known technique for improving the predictive power of ML classifiers in complex markets like Bitcoin [39] [63]. The output of this stage is a structured DataFrame of feature vectors, which becomes the primary input for both the machine learning models in the TITAN pipeline and the factor analysis performed by `Alphalens`.

Following feature generation, the alpha discovery stage validates the predictive power of these features. Here, `Alphalens-Reloaded` is the designated tool. The original `Alphalens` library is a well-established industry standard for analyzing the performance of stock prediction factors [2] [6]. However, the original library is no longer actively maintained, leading to compatibility issues with modern Python versions and data science libraries [3]. Consequently, the `-Reloaded` version is not just an option but a necessity, as it addresses these maintenance issues and ensures the library remains functional within the contemporary software ecosystem [4] [5]. `Alphalens` takes as input the factor values (which could be the z-scores of technical indicators derived from `tsfresh`, or predictions from a simple linear model) and the corresponding historical asset prices [16]. It then produces a detailed suite of performance metrics and visualizations. Key outputs include the Information Coefficient (IC), which measures the correlation between the factor value and the subsequent asset return; quantile-based returns analysis, which groups assets by their factor rank and plots their average future returns; and risk-adjusted performance metrics for each quantile [9] [26]. The results generated by `Alphalens-Reloaded` provide a scientific basis for determining whether a given feature possesses genuine alpha-generating properties.

To complement the factor-specific analysis of `Alphalens`, `Pyfolio-Reloaded` is integrated for broader performance and risk analysis. While originally designed to work seamlessly with the Zipline backtesting library, `Pyfolio-Reloaded` is a versatile tool that can analyze any set of portfolio returns [25]. After a strategy has been backtested, either in the VectorBT or Backtrader stage, its resulting equity curve and trade log can be fed into `Pyfolio-Reloaded`. This library generates a rich set of performance reports,

including cumulative returns, drawdown analysis, turnover, and risk metrics like Sharpe ratio and beta, all presented in a visually intuitive format. This allows for a holistic assessment of a strategy's risk-reward profile beyond what a single factor's IC can reveal. The integration of `Pyfolio-Reloaded` ensures that the final evaluated strategies are not only based on strong alpha factors but also exhibit desirable risk characteristics.

The interplay between these stages is critical. The feature vectors produced by `tsfresh` are the direct input for creating the factor values analyzed by `Alphalens-Reloaded`. The dynamic IC scoring mechanism already present in the TITAN pipeline can leverage the IC time series generated by `Alphalens` as a direct input, allowing the pipeline to adaptively weight factors based on their recent performance. This creates a powerful feedback loop where the research stage continuously informs the decision-making logic of the core trading engine. The following table details the data flow and component responsibilities for this integrated research stage.

Component	Role in Stage	Inputs	Outputs	Integration Logic
Data Management Module	Provides clean, aligned historical data.	External data sources (BMLL, OpenBB, etc.).	Pandas DataFrames of OHLCV/ tick data for global assets.	Acts as the single source of truth, ensuring all research tools receive consistent, high-quality data meeting the 'from 2000' constraint.
`tsfresh`	Systematic time-series feature extraction.	Historical price/ volume data per asset.	DataFrame of feature vectors (features x assets x time).	Automates the creation of a wide array of statistical and domain-specific features from raw time-series data, serving as inputs for both ML models and factor analysis [38] [90].
`Alphalens-Reloaded`	Alpha factor performance analysis.	Factor values (e.g., z-scored features) and historical prices.	Detailed reports on factor performance, including IC time series, quantile returns, and turnover statistics [1].	Consumes factor signals to scientifically validate their predictive power, providing crucial feedback for the TITAN pipeline's dynamic IC scoring.
`Pyfolio-Reloaded`	Portfolio performance and risk analysis.	Equity curves and trade logs from backtests.	Comprehensive performance reports and visualizations (drawdowns, risk metrics, etc.).	Provides a holistic view of a strategy's risk-adjusted performance after backtesting, ensuring strategies are not only profitable but also robust [25].

In summary, this integrated research stage transforms raw data into validated alpha signals. By combining the automated feature generation of `tsfresh` with the rigorous statistical validation of `Alphalens-Reloaded` and the broad performance analysis of `Pyfolio-Reloaded`, the TITAN integration layer establishes a powerful and repeatable process for alpha discovery and factor evaluation. This forms the intellectual core of the entire trading system, feeding high-quality signals into the subsequent stages of strategy validation and portfolio construction.

Stage 2: Multi-Paradigm Backtesting and Strategy Validation

After identifying promising factors and features, the next critical stage is to test them within a simulated trading environment. This backtesting and strategy validation stage is responsible for assessing the real-world viability of trading ideas before committing capital. The integration layer strategically employs three distinct backtesting frameworks —`VectorBT`, `Backtrader`, and `Zipline-Reloaded`—each chosen for its unique strengths and suited to different phases of the validation lifecycle. This multi-paradigm approach allows the TITAN pipeline to balance speed, fidelity, and flexibility, enabling rapid iteration during development and high-fidelity simulation for final-stage validation. The stage consumes the outputs from the feature engineering and alpha discovery stages (factor values, ML predictions) and uses the foundational data module for historical prices.

`VectorBT` is positioned as the high-performance vectorized backtesting engine and should be the default choice for the initial and exploratory phases of strategy development [24] [102]. Built on NumPy and accelerated with C++, `VectorBT` operates on entire arrays of data at once, leading to backtesting speeds that are orders of magnitude faster than event-driven engines [24]. For example, one comparison showed `VectorBT` completing a backtest on 1000 stocks over 10 years in under a minute, whereas `Backtrader` took hours [24]. This extreme speed is invaluable for tasks like purged walk-forward analysis, extensive parameter optimization sweeps, or Monte Carlo simulations, which would be computationally prohibitive with slower engines. `VectorBT` excels at testing quantitative, rules-based strategies where the logic can be expressed in a vectorized manner. Its ability to handle multi-asset portfolios natively and integrate with machine learning libraries like TensorFlow and PyTorch makes it a natural fit for validating the outputs of the TITAN pipeline's machine learning models [24]. The primary limitation of `VectorBT` is that strategies must be fully vectorized, which can be challenging for complex, event-driven logic involving intricate order management or broker-specific behaviors [24].

For strategies that require a higher degree of simulation fidelity, `Backtrader` serves as the event-driven alternative. As an event-driven framework, `Backtrader` processes data point-by-point, executing buy/sell signals and updating portfolio state in discrete chronological steps [101]. This makes it inherently slower than `VectorBT` but significantly more flexible and realistic for modeling complex trading scenarios [47]. It is particularly well-suited for strategies involving options, futures, or other derivatives; intricate order

types (e.g., stop-loss, take-profit); or bespoke execution algorithms that cannot be easily translated into a vectorized format. `Backtrader`'s architecture is designed for maximum customization, allowing developers to build their own indicators, analyzers, and brokerage simulators [24]. While its backtesting speed is a drawback for large-scale screening, this speed is often acceptable for the final validation stage, where the focus shifts from speed to accuracy. The integration layer should reserve `Backtrader` for these complex, high-stakes strategies, using it to simulate slippage, commissions, and other transaction costs with greater precision than `VectorBT` can offer.

`Zipline-Reloaded` completes the trio of backtesting paradigms. A fork of the original Zipline, which was developed for academic research, `Zipline-Reloaded` continues to be actively maintained and documented [24]. It is an event-driven engine that integrates exceptionally well with Jupyter Notebooks, making it ideal for research-oriented tasks that involve iterative hypothesis testing and visualization [24]. While its community activity level is lower than that of `VectorBT` or `Backtrader`, its strengths lie in its rich set of built-in factors and its seamless workflow for researchers who prefer the Jupyter ecosystem [24]. The integration layer can utilize `Zipline-Reloaded` as a third option for generating detailed, publication-quality reports or for research tasks that benefit from its specific set of tools and tight notebook integration. It acts as a bridge between the rapid-fire experimentation of `VectorBT` and the complex simulation of `Backtrader`.

A crucial aspect of this stage's integration is its interaction with the existing TITAN pipeline. The backtesting engines must be able to ingest not only historical price data and the newly created factors but also the pipeline's internal signals. Specifically, the outputs of the Kalman-GARCH filters—namely the Kalman trend estimates and GARCH volatility forecasts—must be programmatically accessible to the backtesters. These signals can be incorporated directly into the strategy logic. For example, a strategy could be designed to only take long positions when the Kalman trend estimate is positive and to adjust position sizing based on the GARCH forecast of near-term volatility. This deep integration ensures that the backtested strategies are not operating in a vacuum but are instead leveraging the advanced regime detection and risk forecasting capabilities already embedded in TITAN. The following table compares the three backtesting engines and clarifies their roles within the integration architecture.

Engine	Paradigm	Primary Use Case	Strengths	Weaknesses	Integration Role in TITAN
`VectorBT`	Vectorized	Rapid, large-scale strategy screening and optimization.	Extremely fast execution speed; excellent for Monte Carlo simulations and purged walk-forward analysis [102]; native multi-asset support [24].	Requires strategies to be fully vectorized; less suitable for complex, event-driven logic.	Default engine for initial and iterative backtesting of factor-based and ML-driven strategies.
`Backtrader`	Event-Driven	High-fidelity simulation of complex, custom strategies.	Maximum flexibility and realism; ideal for strategies with complex order logic, slippage models, and commission schemes [101].	Significantly slower than vectorized engines [24]; steeper learning curve.	Reserved for final-stage validation of complex strategies where simulation fidelity is paramount.
`Zipline-Reloaded`	Event-Driven	Academic research and Jupyter Notebook-based analysis.	Excellent integration with Jupyter notebooks; rich set of built-in factors for research; good for generating detailed reports [24].	Lower community activity; requires manual data handling [24].	Used for research-oriented tasks and generating publication-ready reports within the Jupyter ecosystem.

Ultimately, this multi-paradigm backtesting stage provides a robust and flexible validation framework. By intelligently selecting between `VectorBT`, `Backtrader`, and `Zipline-Reloaded` based on the specific needs of the strategy under test, the integration layer ensures that all trading ideas are subjected to rigorous scrutiny. The ability to incorporate TITAN's proprietary signals elevates the backtesting process beyond simple historical simulation, grounding it in the same advanced analytics that drive the core trading engine.

Stage 3: Advanced Portfolio Construction and Optimization

Once a set of viable strategies and assets has been validated through the backtesting stage, the focus shifts to capital allocation. The portfolio construction stage is tasked with determining the optimal weights for assets or strategies to maximize risk-adjusted returns, adhering to predefined constraints. This stage moves from individual strategy performance to the collective performance of a diversified portfolio. The integration layer maps two powerful open-source libraries, `Riskfolio-Lib` and `PyPortfolioOpt`, to this function. These libraries provide a sophisticated toolkit for portfolio optimization that extends far beyond basic mean-variance analysis, allowing the TITAN pipeline to construct robust, resilient investment portfolios across the diverse asset classes of stocks, forex, crypto, and commodities.

`Riskfolio-Lib` is selected for its comprehensiveness and advanced capabilities. Described as a library with the most features for portfolio optimization, it supports a wide variety of risk measures beyond the traditional variance, including Value-at-Risk (VaR) and Conditional Value-at-Risk (CVaR) [20]. This is particularly relevant for assets like cryptocurrencies, which are known for exhibiting fat-tailed return distributions and extreme volatility events [67] [75]. `Riskfolio-Lib` is built on top of `cvxpy`, a powerful domain-specific language for convex optimization embedded within Python, which gives it immense flexibility in defining custom objective functions and constraints [21]. One of its standout features is its support for hierarchical clustering, which helps in creating more diversified portfolios by grouping similar assets together before optimization [20]. The class-based structure of `Riskfolio-Lib` allows for the creation of a portfolio object that encapsulates all necessary inputs, such as expected returns, the covariance matrix, and the assets themselves, streamlining the optimization process [72]. Given the documented spillover effects between crypto assets and traditional financial markets, the ability of `Riskfolio-Lib` to handle complex risk metrics and diversification heuristics makes it an ideal choice for constructing a truly cross-asset portfolio [73] [96].

`PyPortfolioOpt` serves as a complementary and more accessible tool within this stage. Known for its ease of use and extensive documentation, `PyPortfolioOpt` is an excellent library for implementing classic portfolio optimization techniques [22] [84]. It provides straightforward implementations of methods like the efficient frontier, Black-Litterman model, and Hierarchical Risk Parity (HRP) [84]. While perhaps less flexible than `Riskfolio-Lib` for cutting-edge, non-standard optimization problems, its simplicity makes it highly valuable for prototyping, educational purposes, and for implementing well-understood, robust strategies [20]. The integration layer can leverage `PyPortfolioOpt` for more traditional optimization tasks where its established methodologies are sufficient. For example, it could be used to quickly compute minimum variance portfolios or to apply Black-Litterman blending to combine market equilibrium views with the TITAN pipeline's alpha forecasts [55]. Its utility is enhanced by its ability to handle large-scale problems efficiently, with optimization possible even for thousands of assets [84].

A key advantage of integrating these libraries is their capacity to incorporate the outputs of the existing TITAN pipeline as direct inputs. The GARCH volatility forecasts, which are already a core component of the system, can be used as a highly accurate, forward-looking estimate of risk for the optimization models. Instead of relying on static historical volatility, the optimizers can use the conditional variance predicted by the GARCH model for each asset. This allows for dynamic risk management, where portfolio risk targets can be explicitly constrained. For instance, an optimization problem can be formulated to find

the portfolio with the maximum Sharpe ratio subject to the constraint that the portfolio's overall volatility, as measured by the GARCH forecasts, does not exceed a certain threshold (e.g., 2%) [23]. This tight coupling between the risk forecasting and portfolio construction stages is a hallmark of a sophisticated institutional trading system. The variance-covariance matrix estimation, a known challenge in Modern Portfolio Theory, can also be informed by the GARCH-XGBoost models that combine econometric approaches with machine learning for improved volatility modeling [97] [98].

The data flow into this stage begins with the list of assets and strategies deemed "investable" by the backtesting stage. These candidates become the universe from which the optimizer selects. The inputs to the optimization process are: 1. **Expected Returns:** These can be derived from the alpha scores generated by `Alphalens-Reloaded` or from the predictions of the TITAN pipeline's ML ensemble. 2. **Risk Model:** This can be a covariance matrix estimated from historical returns or, more powerfully, a risk model built using the GARCH forecasts [54]. 3. **Constraints and Objectives:** These define the optimization problem. Objectives might include maximizing the Sharpe ratio or minimizing CVaR. Constraints can include limits on sector exposure, maximum position size, and, critically, risk budget constraints derived from the GARCH model [23].

The output of this stage is a set of optimal portfolio weights. These weights represent the final allocation decision and are passed to the execution layer. The table below summarizes the roles of the two primary libraries in this stage.

Library	Role in Stage	Key Features	Integration with TITAN	Example Use Case
`Riskfolio-Lib`	Advanced, comprehensive portfolio optimization.	Supports multiple risk measures (VaR, CVaR), hierarchical clustering, and custom objectives/ constraints via `cvxpy` [21].	Can consume GARCH volatility forecasts as a risk input or constraint. Ideal for optimizing across mixed-asset classes with complex risk dynamics [20].	Constructing a diversified portfolio of global stocks, bonds, commodities, and cryptocurrencies, accounting for tail-risk and volatility spillovers.
`PyPortfolioOpt`	Accessible and robust portfolio optimization.	Implements classical methods (efficient frontier, Black-Litterman), HRP, and is known for its user-friendly API and documentation [22] [84].	Can be used for simpler optimizations or to prototype strategies quickly. Expected returns can be sourced from TITAN's alpha signals.	Computing a minimum variance portfolio for a basket of US equities or applying Black-Litterman to blend TITAN's factor views with market-implied returns.

By integrating `Riskfolio-Lib` and `PyPortfolioOpt`, the TITAN pipeline gains a powerful and flexible portfolio construction engine. This stage effectively translates the outputs of the research and validation stages—the profitable and statistically significant trading ideas—into a concrete, optimized capital allocation plan, ready for execution in the live market.

Stage 4: Execution Learning via Reinforcement Learning

The final frontier in the research-to-execution pipeline is the application of reinforcement learning (RL) to learn optimal trading policies. This stage, termed "Execution Learning," represents a paradigm shift from rule-based or optimization-based decision-making to policy-based learning. The integration layer maps two prominent RL frameworks, `FinRL` and `Qlib`, to this function. These libraries provide the infrastructure to train agents that can learn to execute trades or manage portfolios in a way that maximizes a defined reward signal, such as long-term profit or a Sharpe ratio. This stage leverages the outputs of the preceding stages to inform the agent's understanding of the market and learns an optimal policy for navigating it.

`FinRL` is a modular and deployment-consistent framework designed specifically for AI-native quantitative trading [10] [12]. Its core innovation is the separation of the financial market environment from the RL agent, allowing for the creation of reusable environments for various tasks like stock trading, order execution, and cryptocurrency trading [11] [29]. The `FinRL-Meta` sub-framework further standardizes these environments, supporting over 30 data sources and providing a benchmark for comparing different RL agents [59]. For the TITAN integration, `FinRL` would be used to construct a custom trading environment. This environment would take historical data from the foundational module as its "universe." At each timestep, the environment would present the agent with a "state," which would be a composite representation of the market conditions. Crucially, this state representation can be enriched with the features engineered by `tsfresh` and the signals from the TITAN pipeline (e.g., Kalman trend, GARCH volatility). The agent then chooses an "action" from its action space (e.g., buy, sell, hold, or a continuous variable representing the position size). The environment calculates the "reward" based on the outcome of that action, for example, the change in portfolio value minus transaction costs. Through countless iterations of this trial-and-error process in the simulated environment, the agent learns a policy (a mapping from states to actions) that maximizes cumulative reward. The trained agent can then be deployed in a live setting to make execution decisions.

`Qlib` offers another powerful platform for applying RL to quantitative investment. Developed by Microsoft Research, the Qlib Reinforcement Learning toolkit (`QlibRL`) is an RL platform specifically tailored for the quantitative finance domain [30]. It provides a comprehensive suite of tools to implement, train, and benchmark RL algorithms within the broader Qlib ecosystem. Similar to `FinRL`, `Qlib` would be used to create a trading environment for training an agent. The state representation for the agent in `Qlib` can be

constructed using the rich set of features and factors available in the Qlib framework. The integration with TITAN would involve feeding the agent's state space with the same signals used by the rest of the pipeline. For example, the agent could be trained to optimize portfolio weights dynamically, taking as input the current portfolio holdings, the GARCH volatility forecasts, and the alpha signals from the factor evaluation stage. The reward function could be designed to encourage stable growth and penalize excessive drawdowns. The key advantage of using `Qlib` is its deep integration with the academic and research communities, providing access to state-of-the-art algorithms and benchmarks. The system can learn to generalize trading policies beyond the offline data it was trained on, a critical challenge in RL for finance [58].

The primary output of this stage is a trained RL agent, essentially a neural network (or other function approximator) that embodies a learned trading strategy. This agent does not contain explicit rules like "if SMA(50) crosses above SMA(200), then buy." Instead, it contains learned weights that allow it to perceive market states and select actions that have historically led to high rewards. The integration layer must create a bridge between this learned agent and the final execution protocol of the TITAN pipeline. When a new market state is observed in real-time, it is fed into the agent's policy network, which outputs an action. This action is then passed to the execution engine to be converted into a concrete trade. This creates a closed-loop system where the agent continuously learns and adapts its policy based on its interactions with the market.

The following table outlines the proposed architecture for the execution learning stage.

Component	Role in Stage	Inputs	Outputs	Integration Logic
Data Management Module	Provides historical and real-time data for training and inference.	Global market data (stocks, forex, crypto, commodities) since 2000.	Time-series data for training the RL agent.	Supplies the raw material for the simulated trading environment in both `FinRL` and `Qlib`.
`tsfresh` / TITAN Signals	Generates the state representation for the RL agent.	Cleaned time-series data, Kalman trend estimates, GARCH volatility forecasts.	A feature-rich state vector for each timestep.	Enriches the agent's perception of the market, allowing it to learn more complex, nuanced strategies.
`FinRL` / `Qlib`	Trains the RL agent in a simulated environment.	Custom-built trading environment with defined state, action, and reward spaces.	A trained policy network (the "agent").	Uses the TITAN pipeline's signals and `tsfresh` features to construct the environment's state space, enabling the agent to learn from the same information as the rest of the system.
Execution Protocol	Deploys the trained agent for live decision-making.	Real-time market data and the agent's output action.	Concrete trade orders sent to the broker.	Acts as the interface between the learned policy and the live market, translating the agent's decision into executable trades.

By incorporating `FinRL` and `Qlib`, the TITAN integration layer pushes the system into the realm of adaptive, learning-based trading. This stage allows the pipeline to evolve beyond static strategies, enabling it to discover and exploit patterns in the market that may be too complex for human-defined rules or traditional optimization. The trained RL agent becomes a dynamic component of the trading engine, capable of adapting its behavior over time to changing market regimes.

Integrated System Architecture and Implementation Roadmap

The culmination of this research is a cohesive, multi-stage architecture for the TITAN integration layer. This design orchestrates a suite of powerful open-source libraries to create a complete research-to-execution system. The architecture is not a monolithic block but a flexible, interconnected system of modules, each responsible for a distinct part of the workflow. The success of this integration hinges on the clear definition of data flows between stages and the strategic selection of tools best suited for each task. This final section synthesizes the proposed architecture and provides an actionable roadmap for its implementation, prioritizing practical, incremental steps.

The integrated system can be conceptualized as a pipeline with five primary stages, fed by a foundational data management module. The flow of information is unidirectional, from raw data to final execution decisions, with critical feedback loops at key junctures.

1. **Foundation: Data Management Module.** This is the bedrock of the entire system. It is responsible for sourcing, cleaning, aligning, and storing data for global stocks, forex, crypto, and commodities, ensuring all datasets meet the 'from 2000' historical constraint . It provides a standardized API for all downstream components, abstracting away the complexity of various data sources [14] [59]. This module handles both batch processing for historical data and streaming for real-time feeds, likely using technologies like websockets and potentially Kafka for scalability [37] [49].

2. **Stage 1: Feature Engineering & Alpha Discovery.** This stage transforms raw data into testable hypotheses. The `tsfresh` library systematically extracts hundreds of features from the time-series data provided by the data module [38]. These features are then used to generate factor values, which are rigorously tested for predictive power using `Alphalens-Reloaded` [3]. The resulting Information Coefficient

(IC) time series and other performance metrics serve as a primary feedback mechanism for the TITAN pipeline's internal logic [26]. `Pyfolio-Reloaded` provides a deeper risk analysis of the factor-based portfolios, ensuring that discovered alphas are accompanied by desirable risk characteristics [25].

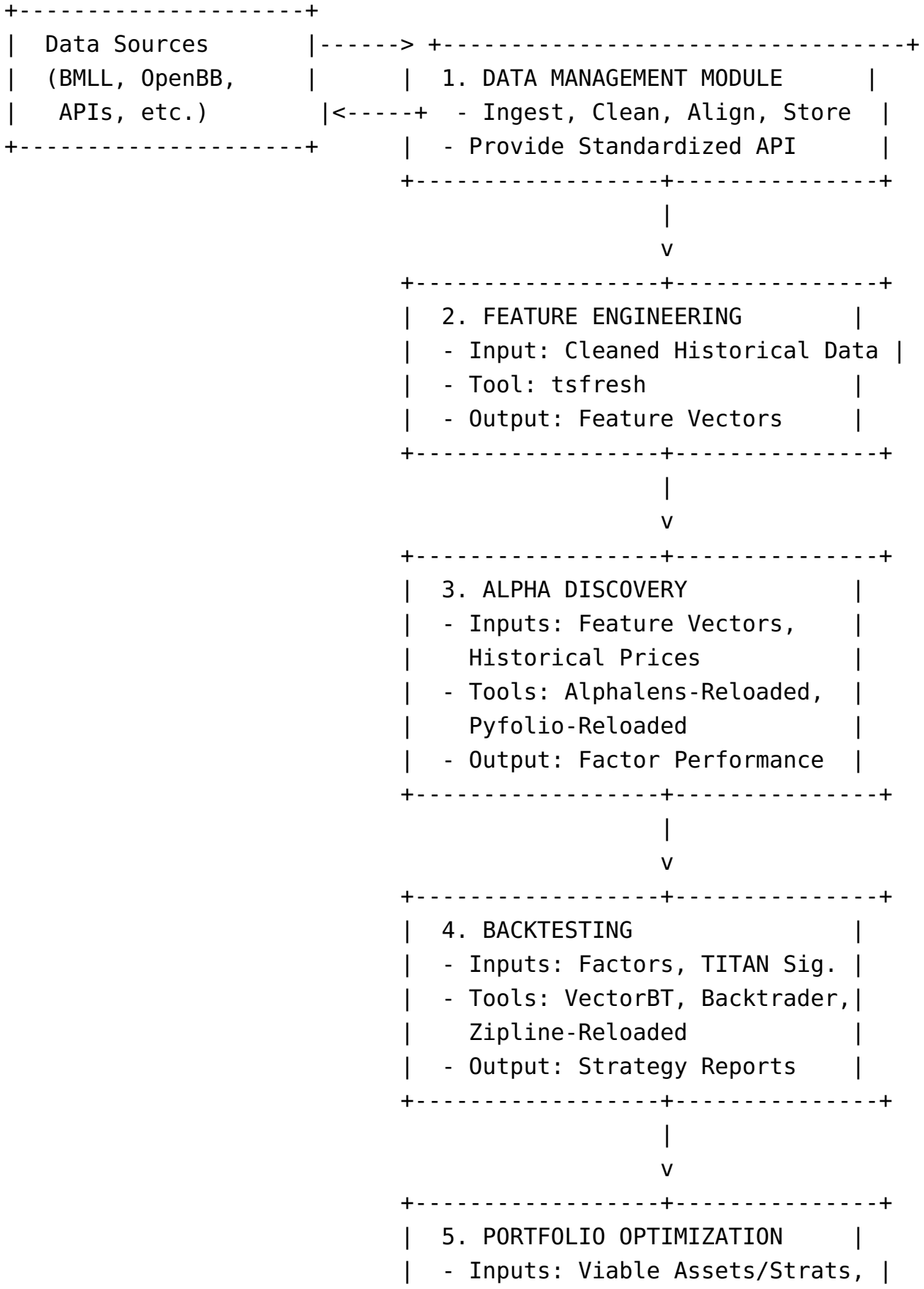
3. **Stage 2: Backtesting & Strategy Validation.** Validated factors are turned into testable strategies in this stage. A multi-paradigm approach is employed: `VectorBT` is used for rapid, large-scale screening and optimization due to its high-speed vectorized computation [24]; `Backtrader` is reserved for high-fidelity simulation of complex, event-driven strategies where realism is more important than speed [101]; and `Zipline-Reloaded` serves the academic research community with its Jupyter-centric workflow [24]. Critically, this stage ingests the Kalman trend estimates and GARCH volatility forecasts from the core TITAN pipeline, allowing strategies to be stress-tested against the system's own risk and regime models.

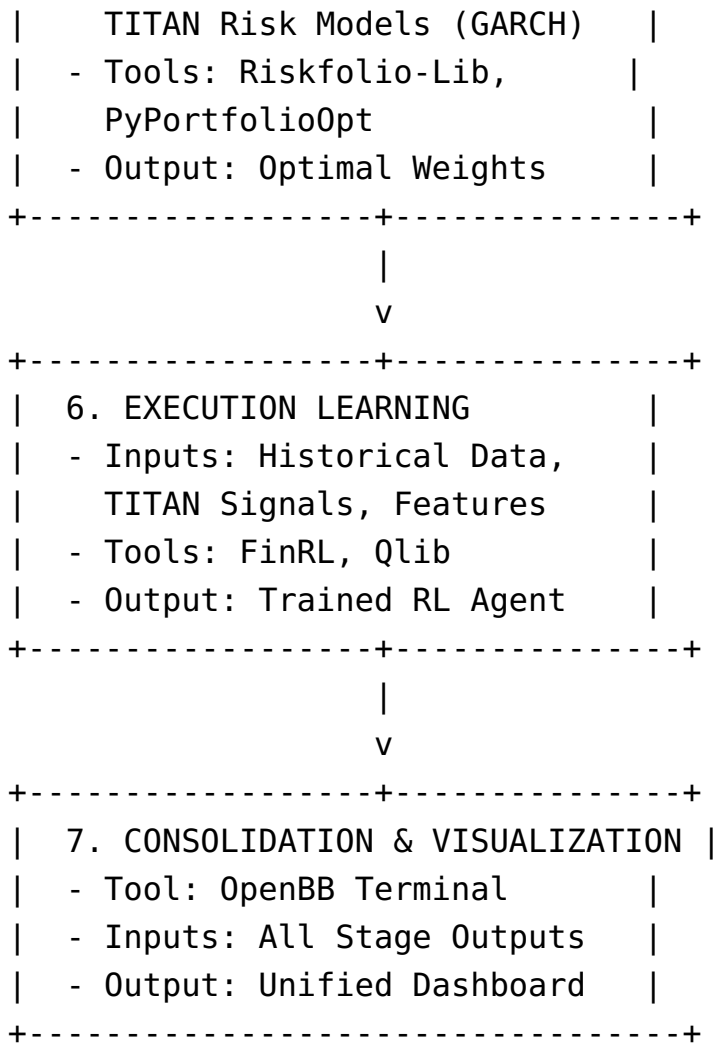
4. **Stage 3: Portfolio Construction & Optimization.** Viable strategies and assets are aggregated here to determine optimal capital allocation. `Riskfolio-Lib` is the primary tool, leveraging its advanced capabilities in handling multiple risk measures (VaR, CVaR) and hierarchical clustering to build robust, diversified portfolios across the four asset classes [20]. `PyPortfolioOpt` provides a more accessible option for traditional mean-variance and Black-Litterman optimizations [22]. Both libraries can consume the GARCH volatility forecasts as a direct input for risk modeling or as an explicit constraint on portfolio risk, closing the loop between risk forecasting and allocation.

5. **Stage 4: Execution Learning.** This forward-looking stage applies reinforcement learning to learn optimal execution policies. Using frameworks like `FinRL` or `Qlib`, an RL agent is trained in a simulated environment [10] [30]. The agent's state representation is enriched with features from `tsfresh` and signals from the TITAN pipeline, allowing it to learn a policy that maximizes a defined reward function. The trained agent then feeds its decisions into the TITAN execution protocol, creating a dynamic, learning-based component of the trading engine.

6. **Consolidation: Model Management & Visualization.** Throughout the process, outputs from all stages are consolidated. `OpenBB Terminal` is the designated tool for providing a unified dashboard for monitoring, visualization, and reporting, pulling together disparate results into a coherent whole [17] [48]. `MlfinLab` can be integrated as a toolbox for additional financial machine learning utilities where needed [18].

The following diagram illustrates the proposed data flow:





Implementation Roadmap:

Given the complexity, a phased implementation is recommended. Immediate guidance takes precedence over theoretical perfection .

- **Phase 1: Establish the Core Loop (Months 1-3).** The highest priority is to build the fundamental research and validation workflow.

```
1.  <strong>Build the Data Management Module:</strong> Focus on sourcing a
2.  <strong>Integrate Feature Engineering:</strong> Connect `tsfresh` to t
3.  <strong>Implement Alpha Discovery:</strong> Integrate `Alphalens-Reloa
4.  <strong>Deploy a Fast Backtester:</strong> Integrate `VectorBT` to tes
```

- **Phase 2: Enhance Fidelity and Scope (Months 4-6).**

```
1.  <strong>Add High-Fidelity Backtesting:</strong> Integrate `Backtrader`
2.  <strong>Incorporate TITAN Signals:</strong> Work with the core TITAN t
3.  <strong>Expand Asset Coverage:</strong> Extend the data module and the
```

- **Phase 3: Add Portfolio and Advanced Learning Layers (Months 7-12).**

```
1.  <strong>Integrate Portfolio Optimization:</strong> Add `Riskfolio-Lib`
2.  <strong>Develop the RL Environment:</strong> Begin work on the `FinRL`
3.  <strong>Build the Visualization Dashboard:</strong> Integrate `OpenBB
```

By following this roadmap, the engineering team can incrementally build a robust and powerful research-to-execution system, starting with a proven core and progressively adding layers of sophistication, ultimately realizing the full potential of the specified open-source libraries within the TITAN pipeline.

Reference

1. Python for Algorithmic Trading Cookbook - O'Reilly https://www.oreilly.com/library/view/python-for-algorithmic/9781835084700/B21323_08.xhtml
2. Alphalens: a Python library for stock factors | PyQuant News posted ... https://www.linkedin.com/posts/pyquant-news_do-you-even-have-alpha-probably-not-activity-7229100030048059394-N5Sk
3. 【Alphalens】使用Alphalens配合Akshare进行双均线因子分析 https://zhuanlan.zhihu.com/p/692641433
4. alphalens请更新到reloaded版本- BigQuant量化交易 https://bigquant.com/wiki/doc/NjWvZDOlK3
5. alphalens-reload - 笨笨和呆呆- 博客园 https://www.cnblogs.com/chentianyu/p/19021174
6. Alphalens 0.4.1post1+8.g3348e92 documentation - AiDocZh https://www.aidoczh.com/alphalens/
7. Alphalens高级应用：3大实战案例教你优化投资策略 - CSDN博客 https://blog.csdn.net/gitblog_01192/article/details/155295483
8. alphalens 因子分析- Heywhale.com https://www.heywhale.com/mw/project/642bcfed34db799a8a12ab9c

9. ML4T - 第7章第6节使用Alphalens进行分析Alphalens Analysis 原创 https://blog.csdn.net/2401_82851462/article/details/152330419
10. FinRL-X: An AI-Native Modular Infrastructure for Quantitative Trading https://arxiv.org/html/2603.21330v1
11. FinRL Ecosystem Tutorial and Introduction (Part II) - 知乎专栏 https://zhuanlan.zhihu.com/p/490702927
12. FinRL-X: An AI-Native Modular Infrastructure for Quantitative Trading https://ideas.repec.org/p/arx/papers/2603.21330.html
13. FinRL-X: An AI-Native Modular Infrastructure for Quantitative Trading https://www.researchgate.net/publication/403072316_FinRL-X_An_AI-Native_Modular_Infrastructure_for_Quantitative_Trading
14. How to Use Tushare as a Data Source for OpenBB - LinkedIn https://www.linkedin.com/pulse/how-use-tushare-data-source-openbb-roger-ye-ro5ac
15. OpenBB Terminal Manual | PDF - Scribd https://www.scribd.com/document/651543826/OpenBB-Terminal-Manual
16. Using Alphalens — Zipline Trader 1.6.0 documentation https://zipline-trader.readthedocs.io/en/latest/notebooks/Alphalens.html
17. Emma Muhleman CPA CFA CQF's Post - LinkedIn https://www.linkedin.com/posts/emmamuhleman_github-openbb-financeopenbb-financial-activity-7258330501650776064-g4DJ
18. Mlfin.py https://mlfinpy.readthedocs.io/
19. Riskfolio-Lib 7.3 https://riskfolio-lib.readthedocs.io/
20. 用riskfolio-lib优化投资配置(附代码) https://zhuanlan.zhihu.com/p/12214876163
21. riskfolio-lib - PyPI https://pypi.org/project/riskfolio-lib/0.0.5/
22. PyPortOptimization: A portfolio optimization pipeline leveraging ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12370148/
23. How to add volatility constraint to this optimization problem using ... https://stackoverflow.com/questions/75487636/how-to-add-volatility-constraint-to-this-optimization-problem-using-pyportfolioo
24. VectorBT Backtrader Zipline 比较- Hopesun - 博客园 https://www.cnblogs.com/hopesun/p/18815644
25. pyfolio-reloaded - PyPI https://pypi.org/project/pyfolio-reloaded/
26. 基于Alphalens做可转债全市场数据的单因子分析（附python代码+ ... https://zhuanlan.zhihu.com/p/710599016
27. 100-Years of Multi-Asset Trend-Following - QuantPedia https://quantpedia.com/100-years-of-multi-asset-trend-following/

28. Global Cross-Market Trading Optimization Using Iterative Combined ... https://www.researchgate.net/publication/390896768_Global_Cross-Market_Trading_Optimization_Using_Iterative_Combined_Algorithm_A_Multi-Asset_Approach_with_Stocks_and_Cryptocurrencies
29. Benchmarking Data-driven Financial Reinforcement Learning Agents https://arxiv.org/html/2504.02281v3
30. Reinforcement Learning in Quantitative Trading - Qlib Documentation https://qlib.readthedocs.io/en/latest/component/rl/overall.html
31. Overview on extracted features - tsfresh - Read the Docs https://tsfresh.readthedocs.io/en/latest/text/list_of_features.html
32. [PDF] The ETS challenges - arXiv https://arxiv.org/pdf/1811.07792
33. Rolling/Time series forecasting - tsfresh - Read the Docs https://tsfresh.readthedocs.io/en/latest/text/forecasting.html
34. Multivariate Time Series Feature Extraction - Kaggle https://www.kaggle.com/code/aledelunap/multivariate-time-series-feature-extraction
35. Time Series Prediction Based on Multi-Scale Feature Extraction https://www.mdpi.com/2227-7390/12/7/973
36. (PDF) Optimizing PyFlink for high-throughput machine learning https://www.researchgate.net/publication/390410591_Optimizing_PyFlink_for_high-throughput_machine_learning_Streaming_feature_engineering_in_banking
37. Building a Robust Real-Time Financial Data Streaming Pipeline https://www.linkedin.com/posts/tegisty-degef-714455209_building-a-robust-real-time-financial-data-activity-7327641884611207170-L5G7
38. Introduction — tsfresh 0.21.1.post0.dev4+g6aa24fd documentation https://tsfresh.readthedocs.io/en/latest/text/introduction.html
39. Multi-Timeframe Feature Engineering for Bitcoin Market Prediction https://www.mdpi.com/2571-9394/8/3/40
40. Mlfinlab PyPI | PDF | System Software - Scribd https://fr.scribd.com/document/431569570/mlfinlab-PyPI
41. [PDF] Investment Portfolio Optimization Based on Modern Portfolio Theory ... https://arxiv.org/pdf/2508.14999
42. [PDF] Discrete Event Modeling and Simulation of Large Markov Decision ... https://theses.hal.science/tel-04103673v1/file/these_barbieri_emanuele.pdf
43. Deep Learning for Stock Price Prediction and Portfolio Optimization https://www.researchgate.net/publication/384707455_Deep_Learning_for_Stock_Price_Prediction_and_Portfolio_Optimization

44. EvoFolio: a portfolio optimization method based on multi-objective ... https://www.researchgate.net/publication/378312152_EvoFolio_a_portfolio_optimization_method_based_on_multi-objective_evolutionary_algorithms
45. Individual Data Lab 2 - Portfolio Construction - Deepnote https://deepnote.com/app/yiteng-shen-eton/Individual-Data-Lab-2-Portfolio-Construction-e385ffe8-447b-437e-9591-4e36bb215818
46. 随笔档案「2025年4月8日」： VectorBT Backtrader Zipline 比较 ... https://www.cnblogs.com/hopesun/p/archive/2025/04/08
47. 有问题，就会有答案 - 知乎 https://www.zhihu.com/en/article/616104522
48. Zipline、VectorBT - Python生态下数据服务提供商： OpenBB https://blog.csdn.net/lonelymanontheway/article/details/160157448
49. How to get ALL (or multiple) pair's historical klines from Binance API ... https://stackoverflow.com/questions/63515267/how-to-get-all-or-multiple-pairs-historical-klines-from-binance-api-in-one-re
50. Algorithmic crypto trading using information-driven bars, triple ... https://link.springer.com/article/10.1186/s40854-025-00866-w
51. Definitive Guide To Pairs Trading | PDF - Scribd https://www.scribd.com/document/642354311/Definitive-Guide-to-Pairs-Trading
52. Portfolio Optimization - Riskfolio-Lib 7.3 https://riskfolio-lib.readthedocs.io/en/latest/portfolio.html
53. Cboe Market Data Services Innovation Spotlight https://www.cboe.com/market_data_services/spotlight/
54. Risk Models — PyPortfolioOpt 1.5.4 documentation https://pyportfolioopt.readthedocs.io/en/latest/RiskModels.html
55. PyPortOptimization: A Portfolio Optimization Pipeline Leveraging ... https://www.researchgate.net/publication/388796868_PyPortOptimization_A_Portfolio_Optimization_Pipeline_Leveraging_Multiple_Expected_Return_Methods_Risk_Models_and_Post-Optimization_Allocation_Techniques
56. [PDF] Return to RiskMetrics: The Evolution of a Standard - MSCI https://www.msci.com/documents/10199/dbb975aa-5dc2-4441-aa2d-ae34ab5f0945
57. Historical Data for Systematic Trading Firms - BMLL https://www.bmlltech.com/industry/systematic-trading-firms
58. [PDF] Learning to Generalize RL Trading Policies Beyond Offline Data https://ojs.aaai.org/index.php/AAAI/article/view/40027/43988
59. [PDF] FinRL-Meta: Market Environments and Benchmarks for Data-Driven ... https://arxiv.org/pdf/2211.03107

60. Hybrid Kalman Filter and Deep Learning Models for Long-Term ... https://www.researchgate.net/publication/389914014_Hybrid_Kalman_Filter_and_Deep_Learning_Models_for_Long-Term_BitTorrent_Traffic_Forecasting
61. 【金融量化】利用ccxt爬取交易所的历史交易烛线图数据信息 https://zhuanlan.zhihu.com/p/577349188
62. FAQ — tsfresh 0.20.1 documentation - Read the Docs https://tsfresh.readthedocs.io/en/v0.20.1/text/faq.html
63. Rolling/Time series forecasting — tsfresh 0.21.0 documentation https://tsfresh.readthedocs.io/en/v0.21.0/text/forecasting.html
64. HRformer: A Hybrid Relational Transformer for Stock Time Series ... https://www.mdpi.com/2079-9292/14/22/4459
65. tsfresh timeseries missing values - python - Stack Overflow https://stackoverflow.com/questions/64238345/tsfresh-timeseries-missing-values
66. Time series forecasting — tsfresh 0.11.1 documentation https://tsfresh.readthedocs.io/en/v0.11.1/text/forecasting.html
67. Volatility persistence in cryptocurrency markets under structural breaks https://www.researchgate.net/publication/343197586_Volatility_persistence_in_cryptocurrency_markets_under_structural_breaks
68. Adaptive Hierarchical Hidden Markov Models for Structural Market ... https://www.mdpi.com/1911-8074/19/1/15
69. A Comparison of Cryptocurrency Volatility-benchmarking New and ... https://arxiv.org/html/2404.04962v1
70. [PDF] Characterizing a Hybrid Batch-Stream Production Workload in ... - HAL https://hal.science/hal-05148331/document
71. [PDF] Asset Allocation Considerations for Cryptocurrency https://advisor.morganstanley.com/seven-brz-group/documents/field/s/se/seven-brz-group/MSSBNA20251001576762_%281%29.pdf
72. Source code for Portfolio - Riskfolio-Lib - Read the Docs https://riskfolio-lib.readthedocs.io/en/latest/_modules/Portfolio.html
73. New Evidence on Spillovers Between Crypto Assets and Financial ... https://www.elibrary.imf.org/view/journals/001/2023/213/article-A001-en.xml
74. 2026 年外汇/ 股票/ 贵金属全市场实时高频金融数据接口横向对比评测 https://cloud.tencent.com/developer/article/2679994
75. Volatility persistence in cryptocurrency markets under structural breaks https://ideas.repec.org/a/eee/reveco/v69y2020icp680-691.html

76. Modeling the Dual Long Memory Property and Structural Breaks https://www.mdpi.com/2071-1050/15/3/2193
77. Time series forecasting — tsfresh 0.10.1 documentation https://tsfresh.readthedocs.io/en/v0.10.1/text/forecasting.html
78. tsfresh · PyPI https://pypi.org/project/tsfresh/0.20.0/
79. Time-IMM: A Dataset and Benchmark for Irregular Multimodal ... - arXiv https://arxiv.org/html/2506.10412v1
80. [PDF] ZeroTS: Zero-shot Time Series forecasting via - OpenReview https://openreview.net/pdf/b1433b99e2adc67f8a1d1cb818a6166b35bb2de0.pdf
81. Inferring financial stock returns correlation from complex network ... https://arxiv.org/html/2407.20380v1
82. 0x0x3f17f1962B36e491b30A40b... https://data.mendeley.com/datasets/gfz25d39fk
83. https://sa.investing.com/pro/pricing/research?entr... https://sa.investing.com/pro/pricing/research?entry=equity_research_top&ticker=8250
84. PyPortfolioOpt性能优化终极指南：用cProfile快速找出投资组合代码瓶颈 https://blog.csdn.net/gitblog_00467/article/details/154107519
85. Optimize Stock Portfolio with PyPortfolioOpt | PDF - Scribd https://www.scribd.com/document/734711517/Easily-Optimize-a-Stock-Portfolio-using-PyPortfolioOpt-in-Python-by-Damian-Boh-DataDrivenInvestor
86. Historical Market Data for Global Stock Exchanges - BMLL https://www.bmlltech.com/market-data-coverage
87. Re(Visiting) Time Series Foundation Models in Finance - arXiv https://arxiv.org/html/2511.18578v1
88. Revisiting Multivariate Time Series Forecasting with Missing Values https://arxiv.org/abs/2509.23494
89. Effects of the irregular sample and missing data in time series analysis https://www.researchgate.net/publication/7257956_Effects_of_the_irregular_sample_and_missing_data_in_time_series_analysis
90. [PDF] tsfresh Documentation https://tsfresh.readthedocs.io/_/downloads/en/v0.12.0/pdf/
91. Detecting Structural Changes in Bitcoin, Altcoins, and the S&P 500 ... https://www.mdpi.com/1911-8074/18/8/450
92. Enhancing Regime Shift Detection Using Unstructured Data - arXiv https://arxiv.org/html/2605.30363v1
93. A forest of opinions: A multi-model ensemble-HMM voting framework ... https://www.researchgate.net/publication/397111020_A_forest_of_opinions_A_multi-

model_ensemble-HMM_voting_framework_for_market_regime_shift_detection_and_trading

94. riskfolio-lib - PyPI https://pypi.org/project/riskfolio-lib/
95. Cboe Cryptocurrency Derivatives Suite https://www.cboe.com/tradable-products/cryptocurrency-derivatives/
96. Diversification for Stock Markets: Commodities or Crypto https://www.researchgate.net/publication/404809240_Diversification_for_Stock_Markets_Commodities_or_Crypto
97. GARCH-XGBoost for Improved Volatility Modelling of the JSE Top40 ... https://www.mdpi.com/2227-7072/13/3/155
98. [PDF] Combining deep learning and GARCH models for volatility and risk ... https://arxiv.org/pdf/2310.01063
99. pyportfolioopt - PyPI https://pypi.org/project/pyportfolioopt/
100. Cboe Product, Newfound Research LLC - Cboe Listings https://www.cboe.com/us/equities/listings/listed_products/issuer_detail/NFRL/
101. 【量化交易】回测引擎设计：事件驱动与向量化 https://quant67.com/post/quant/19-backtest-engine/19-backtest-engine.html
102. 量化开源项目对比Backtrader, VectorBT, Zipline, vnpy, wtpy, qlib ... https://blog.csdn.net/zhangyunchou2015/article/details/147185325